

AUTOTRAQ API Documentation

Base URL

```
http://localhost:3001/api
```

Authentication

All endpoints except `/auth/register` and `/auth/login` require JWT authentication.

Include the token in the Authorization header:

```
Authorization: Bearer <token>
```

Roles

- `admin` - Full access
 - `manager` - Can manage parts, vehicles, approve requests
 - `fulfillment` - Can receive stock and fulfill requests
 - `viewer` - Read-only access
-

Auth Endpoints

POST `/auth/register`

Register a new user.

Request Body:

```
{
  "email": "user@example.com",
  "password": "password123",
  "name": "John Doe",
  "role": "viewer" // optional, defaults to "viewer"
}
```

Response (201):

```
{
  "data": {
    "user": {
      "id": 1,
      "email": "user@example.com",
      "name": "John Doe",
      "role": "viewer"
    },
    "token": "eyJhbGc..."
  }
}
```

POST /auth/login

Authenticate and get JWT token.

Request Body:

```
{
  "email": "user@example.com",
  "password": "password123"
}
```

Response (200):

```
{
  "data": {
    "user": { ... },
    "token": "eyJhbGc..."
  }
}
```

GET /auth/me

Get current authenticated user. **Requires Auth.**

Response (200):

```
{
  "data": {
    "id": 1,
    "email": "user@example.com",
  }
}
```

```
    "name": "John Doe",
    "role": "admin",
    "createdAt": "2024-01-01T00:00:00.000Z"
  }
}
```

Parts Endpoints

POST /parts

Create a new part. **Requires: admin, manager**

Request Body:

```
{
  "sku": "BRK-001",
  "name": "Brake Pad Set",
  "description": "Front brake pads for sedans"
}
```

Response (201):

```
{
  "data": {
    "id": 1,
    "sku": "BRK-001",
    "name": "Brake Pad Set",
    "description": "Front brake pads for sedans",
    "createdAt": "..."
  }
}
```

GET /parts

List/search parts. **Requires Auth.**

Query Parameters: - `search` - Search by SKU, name, or description - `page` - Page number (default: 1) - `limit` - Items per page (default: 20, max: 100)

Response (200):

```
{
  "data": {
    "parts": [...],
    "pagination": {
      "page": 1,
      "limit": 20,
      "total": 100,
      "totalPages": 5
    }
  }
}
```

GET /parts/:id

Get part by ID with fitments and interchange groups. **Requires Auth.**

POST /parts/:id/fitments

Add vehicle fitment to part. **Requires: admin, manager**

Request Body:

```
{
  "vehicleId": 1
}
```

DELETE /parts/:id/fitments/:vehicleId

Remove fitment. **Requires: admin, manager**

Vehicles Endpoints

POST /vehicles

Create a new vehicle. **Requires: admin, manager**

Important: Year must be 2000 or later.

Request Body:

```
{
  "year": 2020,
```

```
"make": "Honda",
"model": "Civic",
"trim": "EX" // optional
}
```

GET /vehicles

List/search vehicles. **Requires Auth.**

Query Parameters: - `search` - Search by make, model, or trim - `year` - Filter by year - `make` - Filter by make - `model` - Filter by model

Interchange Groups Endpoints

POST /interchange-groups

Create interchange group. **Requires: admin, manager**

Request Body:

```
{
  "name": "Civic Brake Pads 2016-2024",
  "description": "Interchangeable front brake pads"
}
```

GET /interchange-groups

List all groups with members. **Requires Auth.**

POST /interchange-groups/:id/members

Add part to group. **Requires: admin, manager**

Request Body:

```
{
  "partId": 1
}
```

DELETE /interchange-groups/:id/members/:partId

Remove part from group. **Requires: admin, manager**

Inventory Endpoints

POST /inventory/locations

Create a location. **Requires: admin, manager**

Request Body:

```
{
  "name": "Main Warehouse"
}
```

GET /inventory/locations

List all locations. **Requires Auth.**

POST /inventory/receive

Receive stock (creates RECEIVE event). **Requires: admin, manager, fulfillment**

Request Body:

```
{
  "partId": 1,
  "locationId": 1,
  "qty": 10,
  "reason": "P0 #12345" // optional
}
```

POST /inventory/correct

Stock correction. **Requires: admin, manager**

Request Body:

```
{
  "partId": 1,
  "locationId": 1,
  "qty": -5, // positive or negative
  "reason": "Physical count adjustment" // required
}
```

GET /inventory/on-hand

Get on-hand quantities. **Requires Auth.**

Query Parameters: - `partId` - Filter by part - `locationId` - Filter by location

Response (200):

```
{
  "data": [
    {
      "partId": 1,
      "locationId": 1,
      "quantity": 10,
      "part": { ... },
      "location": { ... }
    }
  ]
}
```

GET /inventory/events

Get inventory event history. **Requires Auth.**

Query Parameters: - `partId` - Filter by part - `locationId` - Filter by location - `type` - Filter by type (RECEIVE, FULFILL, RETURN, CORRECTION)

Requests Endpoints

POST /requests

Create a request. **Requires Auth.**

Request Body:

```
{
  "items": [
    {
      "partId": 1,
      "qtyRequested": 3,
      "locationId": 1 // optional, specifies source location
    }
  ],
}
```

```
"notes": "Urgent order" // optional
}
```

GET /requests

List requests. **Requires Auth.**

Query Parameters: - `status` - Filter by status (PENDING, APPROVED, FULFILLED, CANCELLED)

GET /requests/:id

Get request details. **Requires Auth.**

POST /requests/:id/approve

Approve a pending request. **Requires: admin, manager**

Valid transition: PENDING → APPROVED

POST /requests/:id/fulfill

Fulfill an approved request. **Requires: admin, manager, fulfillment**

Valid transition: APPROVED → FULFILLED

This creates FULFILL inventory events and decreases on-hand.

POST /requests/:id/cancel

Cancel a request. **Requires: admin, manager**

Valid transitions: PENDING → CANCELLED, APPROVED → CANCELLED

Error Responses

All errors follow this format:

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Human readable message",
    "details": {
      "field": "Error for this field"
    }
  }
}
```



```
}  
}
```

Error Codes

- `VALIDATION_ERROR` (400) - Invalid input
- `UNAUTHORIZED` (401) - Missing or invalid token
- `FORBIDDEN` (403) - Insufficient permissions
- `NOT_FOUND` (404) - Resource not found
- `RATE_LIMITED` (429) - Too many requests
- `SERVER_ERROR` (500) - Internal error